Lecture 06: Objects in JavaScript

Array Sorting:

.sort(): Sorts elements in ascending order by default.
.reverse(): Reverses the order of elements (often used for descending order).

```
const names = ["Rohit", "Mohit", "Aditya"];
console.log(names.sort());
console.log(names.reverse());
```

How .sort() Behaves with Numbers:

By default, the .sort() method considers each element as a **string** and performs a character-by-character comparison.

Comparator Function:

To sort numbers mathematically, a comparator function must be passed into the sort method.

.sort((a, b) => a - b): Sorts elements in ascending order.
.sort((a, b) => b - a): Sorts elements in descending order.

```
const num = [10, 20, 15, 25, 45, 30];
console.log(num.sort((a,b) => a-b));
console.log(num.sort((a,b) => b-a));
```

How the Mathematical Sort Works:

- If the result is negative (a - b < 0), a is placed before b.
- If the result is positive (a - b > 0), b is placed before a.
- If the result is zero, their order remains unchanged.

Why does it consider values as strings?

Because JavaScript arrays are heterogeneous (they can store different data types).

Note: Sorting an array with mixed data types is an extreme, hypothetical scenario that is practically never used in real-world application logic.

Array Flattening:

The .flat(depth) Method: The built-in .flat() method creates a brand-new array with all sub-array elements concatenated into it up to a specified depth. It does not mutate the original array.

```
const arr = [1, 2, [3, 4], [[5, 6]]];
console.log(arr.flat(2));
```

Objects in JavaScript:

Objects are used to store structured information. Instead of relying on numeric indexes like arrays, data is stored using explicitly named **Key-Value pairs**.

Why store data as an Object?

It gives meaning to the data, making it readable and organized.

```
const user = {
   name: "Rohit",
   age: 20,
   city: "Kotdwar",
   email: "negi@gmail.com",
}
```

How is data received from the backend?

It is typically received as an **Array of Objects**.

```
const users = [
   { name: "Rohit", age: 20, city: "Kotdwar" },
   { name: "Mohit", age: 25, city: "Dwarka" }
];
```

Why an array?

Because arrays can be iterated easily over multiple objects.

Note: const can be safely used inside a for...of loop because the block scope treats it as a brand-new variable on every single iteration.

Accessing and Modifying Object Properties:

Dot Notation (.): The standard, preferred way to access or modify known properties.

Bracket Notation ([]): Used rarely, primarily when keys are dynamic or contain special characters.

```
console.log(user.name);
console.log(user["email"]);
```

Why string syntax?

Internally, JavaScript stores all object keys as strings.

Creating a New Property:

```
user.country = "India";
          or
user["country"] = "India";
```

Deleting an Existing Property:

```
delete user.age;
```

Complex Objects:

An object is highly flexible and can contain functions (methods), arrays, and other nested objects.

Object with Methods:

```
const user = {
   name: "Rohit",
   age: 20,
   greet: function() {
      console.log("Hello");
   }
}
```

Object with Arrays:

```
const user = {
   name: "Rohit",
   age: 20,
   hobbies: ["cricket", "music"],
}
```

Nested Object:

```
const user = {
   name: "Rohit",
   age: 20,
   address: {
    city: "Dwarka",
    pincode: 110075
   }
};
```

Built-in Object Methods:

Method	Description	Return Format
Object.keys(ObjectName)	Returns all keys.	1D Array
Object.values(ObjectName)	Returns all values.	1D Array
Object.entries(ObjectName)	Returns both Keys & Values.	2D Array

Looping Over Objects: Keys and Values

Iterating through object properties requires specific looping structures, as objects are not directly iterable in the exact same manner as arrays.

Why iterate over keys and values?

Iteration allows for dynamic access and helps in targeting or filtering selected keys based on specific conditions.

1. The for...in Loop

The for...in loop is designed specifically to iterate directly over the enumerable string properties (keys) of an object.

```
for(const key in user){
   console.log(key, user[key]);
}
```

Limitation:

The for...in loop is generally not recommended for critical object iteration in modern application logic.

- **Prototype Chaining:** It iterates over all enumerable properties, including inherited properties attached to the object's prototype, which can lead to unexpected data extraction.
- **Unpredictable Order:** The specification does not strictly guarantee the order in which properties are iterated.

2. The for...of Loop

The for...of loop cannot iterate directly over a standard object. It must be paired with built-in Object methods that extract the data into an iterable array.

Iterating over Keys only:

```
for(const key of Object.keys(user)){
   console.log(key);
}
```

Iterating over Values only:

```
for(const value of Object.values(user)){
   console.log(value);
}
```

Iterating over Keys and Values simultaneously:

```
for(const [key, value] of Object.entries(user)){
   console.log(key, value);
}
```

Object Reference, Destructuring, and Copying:

1. Object Reference:

Objects are non-primitive data types. Variables store the memory address (reference) pointing to the object, not the actual data values.

```
const obj1 = {
   name: "Rohit"
}
const obj2 = obj1;
obj2.name = "Mohit";

console.log(obj1);
```

2. Object Destructuring:

Destructuring is a syntax pattern that allows specific properties to be unpacked from an object and assigned to standalone variables.

```
const {name, age} = user;
console.log(name, age);
```

3. Copying Objects:

A. Spread Operator (...) – Shallow Copy

The spread operator unpacks the top-level properties of an existing object into a completely new object in memory.

```
const newUser = {...user};
newUser.name = "Mohit";
newUser.hobbies.push("Reading");
newUser.address.pincode = 123456;
console.log(user);
```

Limitation:

It only performs a shallow copy. Nested objects and nested arrays cannot be fully spread. The top-level object is new, but the internal nested structures still share the same memory references as the original object.

B. structuredClone() – Deep Copy

This is a modern, built-in JavaScript method designed to create a completely independent clone of an object.

```
const customer = structuredClone(user);
customer.name = "Shobhit";
customer.address.city = "Dwarka";
customer.address.pincode = 110075;
console.log(customer);
```